SAHAYAK: Add a Static Sign

Collect, train, test and publish a held pose

A static sign can be described by a pose in one frame. This guide explains how to add one while keeping the old signs.

The safe order

1. Back up the CSV, current model and label file.
2. Collect samples of the new pose into the existing CSV.
3. Check the spelling and counts of old and new labels.
4. Train a trial model.
5. Test with fresh examples.
6. Make the model active only if it passes the checks.
7. Test old and new signs locally.
8. Publish only the approved static model.

The static model expects 126 numbers: 2 hands x 21 key points x 3 coordinates.

Before starting

Open VS Code PowerShell. The example project path is:
C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training

Change the first path if your project is in a different folder. Keep your current .venv if it works; do not reinstall packages without a reason.

Step 1. Open the project and back up the files

~~~powershell
$ProjectRoot = "C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training"
if (-not (Test-Path $ProjectRoot)) { throw "Edit ProjectRoot to your real folder." }
Set-Location $ProjectRoot
if (-not (Test-Path ".\.venv\Scripts\Activate.ps1")) { throw ".venv is missing." }
.\.venv\Scripts\Activate.ps1
python --version
python -c "import sys; print(sys.executable)"
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = ".\backups\static_$stamp"
New-Item -ItemType Directory -Force $backup | Out-Null
if (-not (Test-Path ".\data\landmarks_2hand.csv")) { throw "Dataset CSV missing." }
Copy-Item ".\data\landmarks_2hand.csv" $backup -ErrorAction Stop
Copy-Item ".\model\sahayak_twohand.joblib" $backup -ErrorAction Stop
if (Test-Path ".\model\labels_twohand.json") { Copy-Item ".\model\labels_twohand.json" $backup -Force }
Write-Host "Backup saved to $backup"

```
## Step 2. Check the current data
~~~powershell
@'
import csv
from collections import Counter
from pathlib import Path
path = Path("data/landmarks_2hand.csv")
if not path.exists():
raise SystemExit("Dataset CSV not found; check the project folder.")
with path.open(newline="", encoding="utf-8-sig") as f:
rows = list(csv.DictReader(f))
if not rows:
raise SystemExit("Dataset is empty. Stop before training.")
counts = Counter(row["label"].strip().upper() for row in rows if row.get("label", "").strip())
print("Rows:", len(rows))
for label, count in sorted(counts.items()):
print(f"{label}: {count}")
print("All feature columns present:", all(f"v{i}" in rows[0] for i in range(126)))
```
```

```
'@ | python -
```

Every intended old label must still be present. The CSV needs one label column and feature columns v0 through v125. Stop if the file is empty or the columns are wrong.

Step 3. Collect the new pose

~~~powershell
python collect_data.py

```
Follow the collector's instructions. Use one label name and spell it the same way every time. Uppercase words
    with underscores are easier to check, for example PLEASE. Keep the complete hand pose visible. Capture
    varied attempts with natural changes in position and lighting. Do not save many copies of one identical
    frame.

Run the count check in Step 2 again. Check both the number of examples and label names before training.

## Step 4. Train a trial model
~~~powershell
python train_model.py
```

The candidate files should be:
- model\sahayak_twohand_candidate.joblib
- model\labels_twohand_candidate.json

Check the candidate:
~~~powershell
Get-Content .\model\labels_twohand_candidate.json
@'
import joblib
model = joblib.load(r"model\sahayak_twohand_candidate.joblib")
print("Features:", model.n_features_in_)
print("Labels:", sorted(str(x) for x in model.classes_))
'@ | python -
python -m unittest test_sahayak.py

```
The candidate must have 126 features and all intended old labels plus the new label. Do not promote it if an old
    label is missing or an unexpected label appears. A high score on a random split does not guarantee good
    results with a real camera.

## Step 5. Test on fresh examples and promote
Try new examples that were not used for training. Test in different hand positions and lighting. If the result
    is poor, collect better examples and train a new candidate.

Only when the candidate labels and tests look acceptable, run:
~~~powershell
python train_model.py --promote
```

Check the active model and labels:
~~~powershell
Get-Content .\model\labels_twohand.json
@'
import joblib
model = joblib.load(r"model\sahayak_twohand.joblib")
print("Features:", model.n_features_in_)
print("Labels:", sorted(str(x) for x in model.classes_))
'@ | python -

```
## Step 6. Test the local website
~~~powershell
python web_app.py
```

Keep this terminal open while testing. Open your local website and test the new sign and several old signs. Check the predicted label, hand landmarks, moving box and response. Stop the local server with Ctrl+C when finished.

If an old sign now performs worse, restore the backup and investigate before publishing.

Step 7. Publish only the static model

Copy only model\sahayak_twohand.joblib to:
deployment\SAHAYAK-Vercel-Deployment\backend\model\sahayak_twohand.joblib

If labels_twohand.json is used as backend metadata in your setup, update that file too. Back up the deployment copy first. Never copy a candidate model.

Commit only the model file or files that really changed. Push to the branch watched by Render, then wait until the deployment is Live.

Step 8. Check the live service

Open:

https://sahayak-backend-krh1.onrender.com/api/status

Confirm model_ready is true and single_sign_labels includes the new sign. Then open the live website and test the new sign and several old signs.

Do not upload the raw CSV or training database to the public deployment repository. Do not replace the continuous model for a static-only update.

If something goes wrong

- Missing label: check CSV spelling, candidate labels and active labels.
- Wrong feature count: stop before promotion.
- Old signs become worse: restore the old model and review the samples.
- Backend is not ready: wait for Render and check its latest logs.
- Camera drawing looks right but prediction is wrong: check the model and feature input too.
- Command fails: stop and save the complete error before changing files.

Final checklist

[ ] Backup made. [ ] New label spelled correctly. [ ] Old labels kept. [ ] Candidate has 126 features. [ ] Tests and fresh examples checked. [ ] Model promoted only after review. [ ] Old and new signs tested locally. [ ] Only the static model published. [ ] Live API and website checked.